SEARCH
b: max branching factor (finite)
d: depth of optimal solution
m: max depth of search space (finite or infinite)
g(n): path cost from the root of the tree to node *n*
h(n): heuristic (estimated cost) from *n* to goal node
Complete: finds a solution

BFS
Complete: Yes, for finite *b*
Optimal: No - unless step costs are equal
Time: O(b^d)
Space: O(b^d)
Fringe: New node at the end

UCS
Expands the node with least-cost g(n) (cost from root).
Complete: Yes
Optimal: Yes
Time: O(b^(1 + (C*/E)))
Space: O(b^(1 + (C*/E)))
Fringe: Ascending order of g(n)
C* = cost of optimal solution
E = smallest step cost
UCS = BFS when: step cost is a constant or g(n) is a constant and increases with the level: g(n) = depth(n)

DFS
Complete: No - could get stuck as *m* could be infinite.
Optimal: No - doesn't consider step cost & finds the deeper solution first.
Time: O(b^m) - worse than BFS
Space: O(bm) - linear as we remove node from mem
Fringe: New node at the front

Depth-limited search DLS
DFS with a depth limit *l*
Complete: Yes - search depth is always finite
Optimal: No - doesn't consider step cost
Time: O(b^l)
Space: O(bl) - drop from memory after seein all desc
Hard to choose good value for l

Iterative deepening search (IDS)
DLS that tries every value of l.
Combines benefits of DFS (low mem.) and BFS (complete and optimal [in some cases]).
Repeated nodes are close to the root - only 11% more.
Complete: Yes
Optimal: No - unless equal step costs
Time: O(b^d)
Space: O(bd)

Greedy
Expands the node with the smallest h(n) value.
Complete: Yes, for finite m
Optimal: No
Time: O(b^m)
Space: O(b^m)
Fringe: smallest heuristic value at front
Greedy = UCS if h(n) = g(n)

A* search
A* evaluation function: f(n) = g(n) + h(n)
Complete: Yes
Optimal: Yes iff an admissible heuristic
Time: O(b^d)
Space: exponential, keeps all nodes in memory
A* = UCS if h(n) = 0.
A* = BFS when f(n) = depth(n)

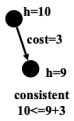
Hill-climbing
Generate successors of n. If child is better, go to child and repeat. If not better, stop: local or global min found. Do not backtrack.
Result is dependent on where you start - can restart.
Complete: No - it can get stuck in local max/minima, plateaus, or ridges, and therefore never find a solution.
Optimal: No - same as above, might stop at suboptimal solution without finding the global max/minimum.
Space: O(b) - only stores current and neighbours.

Simulated annealing
Similar to hill-climbing but selects a random successor instead of the best successor using probability function
e.g. p = exp(val(parent) - val(child))/T
The random walk step (accepting bad children with some probability) can help escape bad local minima.
If T decreases slowly enough → global minima can be found = complete, optimal but time consuming.

Beam-search
Generate all successors of given states. Keep only *k* best states. Repeat.(A* use: restrict fringe to size k)

HEURISTICS
A heuristic is admissible if for every node *n*: h(n) ≤ h*(n) where h*(n) is the true cost to reach a goal from n.
i.e. admissible heuristic = optimistic, relax problem to make

For 2 admissible heuristics h1 and h2, h2 dominates h1 if for all nodes h2(n) ≥ h1(n).
You want to use a heuristic that is the most dominant, but still optimistic. A* expands less nodes w dom heuristics



A heuristic is consistent (non-decreasing)
if for all parent-child ni-nj pairs, h(ni) < cost(ni, nj) + h(nj).

Consistent ⇒ admissible

WEEK 4: GAME PLAYING

Minimax
Generate game tree
Evaluate utility of the terminal states (leaf node)
Recursively compute minimax value up the tree
At root select best move for starting player (usual MAX)

Implemented as DFS
Optimal: Yes - assumes each player will play optimally, but if MIN does not then MAX does even better.
Time: O(b^m) - as DFS
Space: O(bm) - as DFS

Alpha-beta pruning
Alpha = best val for max so far
Beta = best val for min so far
If alpha > beta, prune.

WEEK 5: INTRO TO ML

Overfitting: high training acc, low testing acc.
Reinforcement learning: score instead of values
Lazy learning: KNN
Eager learning: 1R, DTs, Naive Bayes, NNs, SVM

K-Nearest Neighbours
Finds the nearest training data example(s) and returns the class associated with it.

Need to normalise the attributes, usually 0-1 norm:
ai = (vi - min(vi)) / (max(vi) - min(vi))

Time: O(training examples * dimensions)
Memory: O(training examples * dimensions)

Manhattan distance
D(A,B) = |a_1 - b_1| + … + |a_n - b_n|
Euclidean distance
D(A,B) = sqrt((a_1 - b_1)^2 + … + (a_n - b_n)^2)
Minkowski distance
D(A,B) = (|a_1 - b_1|^q + … + |a_n - b_n|^q)^1/q
Nominal distances are 1 if vals are different.

Missing values
Nominal: if one or both are missing = 1
Continuous: if one missing ⇒ max(otherval, 1-otherval).
If both missing = 1.

PROS
Larger k = more robust, but longer time.

CONS
Sensitive to value of *k*. Usually k ≤ sqrt(training egs)
Not good for high-dimensional data as "nearness" is an ineffective measure in higher dimensions ⇒ feature selection.

Weighted Nearest Neighbours (mostly in regression)
Weight each neighbour s.t. closer neighbours count more than distant neighbours:
w_i = 1 / D(X_i, X_new)

1R Algorithm
Finds the attribute that "best" classifies the data by itself.
For each attribute:
  For each val in attribute:
    Count frequency of each class
    Rule: val → most frequent class
    Calculate error for each val
  Calculate total error rate for attribute
Choose the rule with the smallest total error rate.

WEEK 6a: NAIVE BAYES

Using the assumption of conditional independence, we assume the features X1, …, Xn are conditionally independent of each other given Y.
= P(X1 = x1 | Y = y) * P(X2 = x2 | Y = y) * … * P(Xn = xn | Y = y)

This is why it's NAIVE bayes: unrealistic assumptions
a) assumes all attributes are conditionally independent of each other given the class
b) assumes all attributes are equally important

Laplace Correction
To avoid zeroes in the above expression, add 1 example of each value of each attribute for each class. This results in adding 1 to the numerator and *k* to the denominator, where *k* is the number of attr values.

Handle numerical attributes
Use normal distribution for that attribute and class.

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

PROS
Simple calculations due to assumption of independence
Excellent computational complexity
Robust to isolated noise as they are averaged

CONS
Correlated attributes reduce the power of classifier.
Normality assumption might not be true for cont. data.

WEEK 6b: EVALUATING CLASSIFIERS

Holdout (1) ≤ K-Folds (k) ≤ Repeated k-folds (kr)

$$\text{Precision} = \frac{TP}{TP+FN}$$

"What proportion of positive identifications was actually correct?"

$$\text{Recall} = \frac{TP}{TP+FN}$$

"What proportion of actual positives was correctly identified?"

$$F1 = \frac{2PR}{P+R}$$

Harmonic mean of precision and recall.

WEEK 7: DECISION TREES
Select best attribute to split on.
Create a branch for each possible attribute value.
Split the dataset on the branches.
Repeat until stopping criteria satisfied.

Stopping criteria
1. All examples in the subset have the same class.
⇒ Make a leaf node corresponding to this class.
2. All examples in the subset have the same attribute values but different classes (noise in data)
⇒ Make leaf node with majority class of subset
3. Subset is empty
⇒ Make leaf node labelled with majority class of parent's subset

Finding the best attribute
We want to measure the "purity" of each node: a pure node contains only yes or no entries and does not need to be split further.

Entropy
One measure of purity is entropy (unit: bits).

$$H(S) = -\sum_i P_i \cdot \log_2 P_i$$

H = [0, 1].
Smaller the entropy, the less disorder, and the more pure the set.

Information gain
Measures the reduction in entropy (shift towards purity) caused by using specific attribute A to partition the set.
We want the attribute with the largest info gain.

$$Gain(S|A) = H(S) - H(S|A)$$
$$Gain(S|A) = H(S) - \sum_{j \in val(A)} P(A = v_j) H(S|A = v_j)$$

Decision trees try to perfectly classify the training data, so we need to prune them to prevent overfitting.

Tree pruning
Subtree replacement
Replace a subtree with a leaf node if the subtree's performance on validation data is worse than that of a single leaf node.
Subtree rising
Replace a subtree with its most frequent class or value if it improves the tree's performance, effectively "rising" the subtree up to a higher level in the tree.

Rule pruning
Convert the decision tree into a set of rules for each path from the root to the leaves, then prune individual rules by removing conditions that do not improve accuracy, simplifying the model.

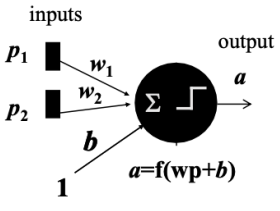
Gain ratio
Reduces the bias of Information Gain towards attributes with many distinct values, promoting more balanced splits. "Entropy w.r.t. attribute."

$$\text{GainRatio} = \frac{Gain(S|A)}{SplitInfo(S|A)}$$
$$\text{SplitInfo} = -\sum_i \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}$$

PROS
Easy to understand, visualise, and interpret
Requires little data preparation

CONS
Very prone to overfitting
Bias towards dominant classes

WEEK 8: PERCEPTRONS



$$a = f(n) = \begin{cases} 1 & if \quad n \geq 0 \\ 0 & if \quad n < 0 \end{cases}$$

The decision boundary is always perpendicular to w.

Perceptron learning rule
Start with a random set of weights (a random decision boundary).
Check each example against the boundary and update the weights until the examples are correctly classified OR until *k* epochs have passed.
1 epoch = 1 pass through the whole training set.

If t = 1 but a = 0, w_new = w_old + p; b += 1
If t = 0 but a = 1, w_new = w_old - p; b -= 1

PROS

CONS
Only works for linearly separable data.
Doesn't find an "optimal" line; just *a* line.

WEEK 10a: SVM
Want to find the maximum margin hyperplane given a set of training data. The margin is the total separation between the DB and the closest examples. Support vectors are the data points closest to the DB, lying on the margin. In SVM, all training examples are at least a distance of 1 away from the DB.

We want the maximum margin because:
a) reduce the model's likelihood of overfitting to the training data, which improves its ability to generalise to unseen data
b) robust: less sensitive to variations in the training data

Decision rule
For an unknown vector u,

$$\vec{w} \cdot \vec{u} + b \geq 0 \implies +$$

Constraints

$$y_i(\vec{w} \cdot x_i + b) - 1 \geq 0$$

where y_i = +1 for positive samples or y_i = -1 for negative samples.

i.e. in SVM, all training examples are at least a distance of 1 away from the DB.

Maximisation
We want to maximise the width of the margin, which can be represented as 2 / ||w||
So, we want to minimise ||w|| ⇒ minimise ½ ||w||^2.

Using Lagrange multipliers, we find that:

$$\vec{w} = \sum_i \lambda_i y_i \vec{x_i}$$

and

$$L = \sum_i \lambda_i - \frac{1}{2} \sum_i \sum_j \lambda_i \lambda_j y_i y_j \vec{x_i} \cdot \vec{x_j}$$

We want to maximise this equation i.e. find lambda i and j that maximise the equation.**Only SV's will have non-zero lambda.**

Then, our decision rule becomes:

$$\sum_i \lambda_i y_i \vec{x_i} \cdot \vec{u} + b \geq 0$$

It is dependent on the dot product of pairs of training vectors.

The weight vector is a row vector. Each entry corresponds to entries in a specific attribute (one column per feature).

Non-linear SVM (Kernel)
All data that is not linearly-separable becomes linearly-separable in a space with enough dimensions.
We can transform SVM to find non-linear boundaries by transforming its original feature space to a new space in higher dimensions, phi.

→ Problem: risk of overfitting if space has ~N dimensions, where N is number of rows in training data.

→ Problem: computationally expensive – first have to computer features in higher dimensions then calculate their dot products.
→ Solution: use Kernel function which computes the dot product of vectors u and v in the higher dimension without ever calculating phi(u) and phi(v).

$$K(\vec{u}, \vec{v}) = (\vec{x} \cdot \vec{y} + 1)^p$$

WEEK 10b: ENSEMBLES
Ensembles work well when (a) base learners are independent of each other; (b) base learners are reasonably accurate.

Bagging – "Bootstrap Aggregating"
Given a dataset D with n examples:
1. Create m bootstrap samples. A bootstrap sample D' from D contains n randomly chosen rows from D (with replacement).
2. Build a classifier on each sample.
3. To predict a new example, get the prediction from each classifier and return the majority prediction.

PROS
Reduction of variance: by averaging several models, the variance of the final prediction is reduced.
Improves accuracy from single classifier.
Robust, as each individual model in the ensemble only sees a subset of the data, smoothing out peculiarities.

CONS
Does not reduce bias (as compared to boosting).

Boosting
Boosting builds models sequentially by focusing on training examples that previous models misclassified. For this we use a weighted training set: each row has a weight associated with it. A higher weight means the row was more difficult to classify, and is more likely to be in the training set for the next classifier.

ADABOOST
Initialise all training examples to have equal weight 1/n.
Train an initial classifier h1.
For each row, assign e_i = 0 if h1 classified correctly, or e_i = 1 if h1 classified incorrectly.
Calculate the weighted sum of the classifier:

$$\epsilon_i = \sum_{j=1}^{n} p_i(\vec{x_j}) e_i(\vec{x_j})$$

Calculate the adjustment term:

$$\beta_i = \frac{\epsilon_i}{1-\epsilon_i}$$

Then adjust the weights/probabilities of the *correctly classified examples* as:

$$p_{i+1}(\vec{x_j}) = p_i(\vec{x_j})\beta_i$$

Then perform normalisation.

This results in the weights of misclassified examples increasing and the weights of incorrectly classified examples decreasing.

The K classifiers are combined **using a weighted vote based on how well the classifier performed on the training set.**

PROS
Reduces bias as it learns from previous models sequentially.
Improved accuracy, even for weak learners.

CONS
Sensitive to noise and data, as boosting continuously tries to correct misclassifications in the data.
Longer training times as cannot be parallelised.

Random Forest
Two ideas: bootstrap sampling and random feature selection.
1. Create bootstrap samples.
2. Train a decision tree on each sample using random feature selection: only a random subset of features is considered for splitting at each node. Usually sqrt(total features) or log_2(total_features). Each tree is usually not pruned to allow for adapting to noise in sample.
3. Classify a new example by majority vote.

PROS
Robust to overfitting: ensemble nature helps average out biases, esp when using large number of trees.
Usually outperforms a single DT.
Faster than AdaBoost whilst giving similar accuracy.

CONS
Complexity of model leads to large model size and significant memory consumption.

WEEK 12:Clustering
In each cluster: similar within (cohesion) and dissimilar across (separation)
Centroid: middle of cluster
Medoid: middle data point of cluster
Single link: Smallest distance
Complete link: largest distance

DB index: heuristic for clusters considering cohesion and separation (pairwise comparison)

$$DB = \frac{1}{k}\sum_{i=1}^{k} \max_{j,j\neq i}\left[\frac{dist(\mathbf{x},c_i)+dist(\mathbf{x},c_j)}{dist(c_i,c_j)}\right]$$

dist(**x**,ci) = mean.sq dist of points from centre
K = number of clusters
Max gives worst pairing: low sep/cohesion

Partional: ONE set of clusters made. K may or may not be specified
Hierarchical: nested set of clusters

K means: iterative, distance based, requires k
Choose k examples as centroids
Form k clusters - assign to closest centroid
After all examples assigned (epoch):
Recompute centroid, stop if unchanged, else repeat

Typical k = 2 to 10
Normalising is important
Centroid can be any point (ok if not a data point)
Change nominal to numeric
#epochs <<<<< points
Sensitive to starting points

Space complexity: modest
Time complexity: expensive -> bad for large data sets

PROS:
Simple + popular
Relatively efficient: fast convergence in modest space
Not sensitive to order of input

CONS
Not optimal
Sensitive to initialisation
Bad w outliers
Best k unknown
Interpreting resulting cluster may be hard (not only kmeans)

Nearest neighbour: used in dynamic data
Items are iteratively merged into clusters

First item is a cluster
New item: either merged or is a new cluster based on threshold (dist<threshold -> merge!)
Sensitive to order of input

Hierarchical
Creates a tree -> dendogram
Root: all in 1 cluster
Leaf: each is a cluster
Level: merging point of clusters
Doesnt req k to be specified

Divisive: top down - opp of agglomerative

Agglomerative: bottom up
Start from singular cluster to root
Based on distance
Uses threshold (dist < d = merge)
Threshold starts small and increments at each level

1) d = 0
2) distance matrix computed
3) each point = cluster
4) repeat:
d++ or d+=n
Merge w dist <= d
Update distance matrix (KEY STEP)

Time complexity: O(n^3)
Can be n^2logn if sorted
Space complexity: O(n^2)

Single link:
If at least 2 points are close -> merge
Others may be far - wont affect cluster
May have unrelated points

Complete link:
Largest distance used so all have to be close
Compact clusters
More robust to noise and outliers

MULTILAYER PERCEPTRONS:

Architecture
Has >= 1 layers of neurons
Feed forward network: input from prev layer only
Fully connected
Randomly initialise weights
Uses differentiable transfer function (non input neurons)

Too many hidden layers and neurons leads to overfitting. Opposite is underfitting.
It does **not** guarantee that the error will be reduced at each epoch since we can reach a local minimum
Backpropagation error threshold: heuristically 0.1

Universality of backpropagation: any continuous function can be approximated with an NN with one hidden layer (Cybenko's theorem), and discontinuous ones can be represented by those with two hidden layers

Backpropagation step:
Calculate all netj

$$net_j = \sum_k w_{kj} o_k + b_j$$

Then calc all oj by using transfer function (sigmoid/tanh)

Backwards pass:
Calculate
δ for all output neurons

$$\delta_q = (d_q - o_q)\cdot f'(net_q)$$

f'(net) for sigmoid = oq (1-oq)
Calculate change in weight and new weights

$$\Delta w_{pq} = \eta \cdot \delta_q \cdot o_p$$

New = old + change
Calculate change in bias and new bias for output

$$\Delta b_6 = \eta * \delta_6 * 1$$

Calculate
δ for all hidden neurons

$$\delta_q = f'(net_q)\sum_i w_{qi}\delta_i$$

Calc new weights and bias

Overfitting: Occurs when the training examples are noisy (meaningless, corrupted), the training dataset is small, or the number of weight parameters in the NN is bigger than the number of training examples.

Prevent overtraining with validation set: eventually acc decreases/error inc - stop and use best until there

Speed up convergence by inc. learning rate but if its too big we might overshoot global min

Smooth out trajectory by getting avg weight update.
Curr update depends on prev update

$$\Delta w_{pq}(t) = \eta\delta_q o_p + \mu(w_{pq}(t) - w_{pq}(t-1))$$

Miu = 0.6 - 0.9

PROS:
Regression and classification supported
Decision boundary for linear and non linear
Universal approximators

CONS:
Slow - needs many epochs
Fully connected - too many params (weight)
Needs large dataset
Needs feature engineering
May not always fnd good sol
Sensitive to start and #hidden laters

CONS of GRADIENT DeCENT
Needs momentum to be fast since LR can increase vanishing gradient problem – the weight changes for the lower levels are very small; these layers learn slower than the higher hidden layers

DEEP LEARNING

Neural networks with >1 hidden layers that are capable of making accurate, data-driven decisions without humans pre-selecting features.
Features can be selected automatically, and training data may even be unlabelled followed by supervised learning.

Why >1 HL - cybenko theorem says that approximation exists w 1 HL but we dont know how to make or if it is optimum (most effective)

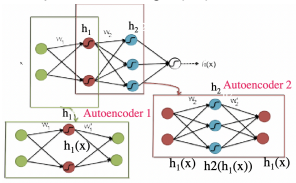
STacked Autoencoders
Unsupervised learning
Target neurons = input
>=1 hidden layer - learns input layer (interest)

Usually #neuron in hidden < input -> compression

Sparse autoencoder: # hidden > input so no compression but is still useful
Denoising: random data points removed. AE learns robust features. Generalizable
Stacked: helps with hierarchical learning (strokes > features > digits)

PROS:
Automatically learns features - good for sensory

Uses of AE:
Dimension reduction, input to other ML/NN, encryption
Used in pretraining of NN - to get weight vector
1 ae per layer used
Use input to learn weight (w1) - discard w1'



Results from hidden h1(x) put into AE2 to learn W2
Because h1(x) is learned version of og data
Og data -> AE1 -> h1(x), w1 -> ae2 -> h2(h1(x)),w2

CONVOLUTIONAL NEURAL NETWORK
Can recognize patterns with high variability.
Designed for images
Robust to distortion & shifting (geom transformation)

CNN is **not** fully connected - only connected to adjacent pixels (patch of image)
Forward pass - missing connections weight = 0
Backward pass = no need to calculate gradient for missing

Weights are shared on different parts of image(filter)
Filter is applied to different positions of input - convolutional layer
Convolution - sliding window applied to matrix
Corresponding elements are multiplied and summed
Makes feature map
Filter: detector of particular shape -vals high if present

Max Pooling takes max value from feature map of neurons from convolutional layer
Also called sub sampling because it summarises input
This makes it invariant to shift because of shared weights
This is important because images and sound have distortion

Local Contrast Normalisation
Controls brightness and variance

Dropout in CNN is used (in fully connected) layers to prevent overfitting
At each iteration of back propagation random neurons are set to 0
while testing we dont drop neurons but scale the weights
Dropout forces CNN do be less dependant on certain neurons - robust to noise

Whole thing can be repeated many times where the output of max pulling is put into another convolutional layer
There are one or two fully connected layers in the end
For coloured images - multi channel inputs (RGB)
Weights are shared within channels but not across
